

Numpy

L'environnement Conda:

Créer un **environnement Conda** présente plusieurs avantages, en science des données, machine learning et développement Python en général. Conda vous permet d'avoir vos propres environnements de travail. Ainsi vous pouvez installer les versions des packages dont vous avez besoin pour mener à bien vos projets.

```
conda create --name myenv python=3.11
conda init bash
conda init zsh
source ~/.bashrc
source ~/.zshrc
conda activate myenv
conda install numpy
```

Exercice 1:

Sans rentrer les coefficients un à un, créer les matrices:

$$A = \begin{pmatrix} 5 & 3 & 3 \\ 3 & 5 & 3 \\ 3 & 3 & 5 \end{pmatrix}, B = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 \end{pmatrix}, C = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{pmatrix}, D = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 1 & 2 & 3 & 4 \\ 1 & 2 & 3 & 4 \\ 1 & 2 & 3 & 4 \end{pmatrix}$$

Utiliser : ones, eye, range, reshape

Exercice 2:

On considère la matrice:

$$B = \begin{pmatrix} 1 & 1 & 0 \\ 1 & -1 & 1 \\ 1 & 0 & 1 \end{pmatrix}$$

a- Vérifier que B est inversible en utilisant la notion du rang:

(`numpy.linalg.matrix_rank(B)`)

b- Calculer son inverse B_inv (`numpy.linalg.inv`)

c- Résoudre le système :

$$\begin{aligned} x + y &= 1 \\ x - y + z &= 3 \\ x + z &= -2 \end{aligned}$$

Exercice 3:

Résoudre, avec Numpy, les systèmes d'équations linéaires suivant d'inconnues x,y et z:

$$1- \begin{cases} x + y + z = 1 \\ x - 2y + z = 0 \\ 2x - y + z = 2 \end{cases}$$

$$2- \begin{cases} x + y + z = a \\ x - 2y + z = b \\ 2x - y + z = c \end{cases} \quad \text{avec} \quad \begin{cases} 3a + b - c = 3 \\ a + b + c = 3 \\ a - 2b + 2c = 1 \end{cases}$$

Exercice 4:

Que vaut le produit d'un vecteur colonne $\begin{pmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{pmatrix}$ par le vecteur ligne $(1 \quad \dots \quad 1)$?

En déduire une ligne de commande créant la matrice:

$$\begin{pmatrix} 1 & 1 & \dots & 1 \\ 2 & 2 & \dots & 2 \\ \vdots & & & \\ 10 & 10 & \dots & 10 \end{pmatrix}$$

Exercice 5:

Pour cet exercice, on prend $n = 1\,000\,000$. Calculer

$$S = \sum_{i=0}^n i$$

Avec et sans utilisation de numpy. Chronométrer le temps nécessaire pour le calcul précédent, par exemple en utilisant `time.perf_counter()`.

1ère méthode:

1- $t_0 = \text{time.perf_counter}()$

2- Créer une liste contenant les valeurs $0, 1, \dots, n$ (utiliser range)

3- $t_1 = \text{time.perf_counter}()$

4- Calculer S en utilisant une boucle simple

5- $t_2 = \text{time.perf_counter}()$

6- afficher $t_1 - t_0$, $t_2 - t_1$ et $t_2 - t_0$

2ème méthode:

- 1- $t_0 = \text{time.perf_counter}()$
- 2- Créer une liste L contenant les valeurs $0, 1, \dots, n$ (utiliser range)
- 3- Créer un numpy array V à partir de la liste L
- 4- $t_1 = \text{time.perf_counter}()$
- 5- Calculer la somme des éléments de V en utilisant la fonction (V.sum())
- 6- afficher $t_1 - t_0$, $t_2 - t_1$ et $t_2 - t_0$

Conclure !

Voci un exemple:

```
import time

# Démarrage du chronométrage
debut = time.perf_counter()

# Code dont on veut mesurer la durée
somme = 0
for i in range(1_000_000):
    somme += i

# Fin du chronométrage
fin = time.perf_counter()

print(f"Durée de l'opération : {fin - debut:.6f} secondes")
```

Exercice 6:

Ecrire une fonction qui prend en argument 3 tableaux numpy et qui renvoie un nouveau tableau numpy contenant toutes les lignes des trois matrices alternativement. Cela suppose que les trois matrices données en entrée à la fonction sont de même taille.

Votre fonction d'abord vérifier la cohérence des tailles des trois matrices avec de procéder à la concaténation.

Exercice 7: Ecrire une fonction f(A) qui prend en paramètre un tableau Numpy A (n'importe quelle taille, n'importe quelle nombre de dimensions) et qui renvoie un tableau B qui est une copie de A dans laquelle les nombres <0 ont été remplacés par 0 et les nombres >1 ont été remplacés par 1. Vous pouvez faire A.copy() pour obtenir une copie de A.

Ecrire deux versions : la première à base de simple boucles et la seconde à base de fonctionnalités numpy.

Pour mesurer les temps nécessaire créer un tableau 10000×10000 et la fonction `time.perf_counter()`

Rappel : A[condition] évalue la condition pour chaque cellule. A[condition]=5 met 5 dans toutes les cellules où la condition est vraie. A[A>5]=5 met 5 dans toutes les cellules dont la valeur est supérieure à 5.

Exercice 8:

Ecrire une fonction qui prend en argument un tableau (numpy) et qui remplace chaque valeur par sa valeur absolue. Proposer deux versions:

- 1- la premier consiste en l'utilisation de boucles (python)
- 2- La second n'utilise que des fonctionnalités numpy

Exercice 9:

Ecrire une fonction qui prend en argument deux numpy.ndarray et qui renvoie le pourcentage de valeurs identiques dans les deux tableaux.

- 1- sans numpy
- 2- avec numpy

Exercice 10:

Une matrice de deux dimensions peut stocker des individus (1ère dimension) et leurs caractéristiques (2ème dimension). Cependant très souvent il est souhaitable de normaliser les données avant de les utiliser. On peut normaliser chaque individu par la moyenne et l'écart-type :

$$\hat{x}_i = \frac{(x_i - m)}{et}$$

$$m = \frac{1}{n} \sum_{i=1}^n x_i$$

$$et = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - m)^2}$$

Etant donnée une matrice contenant des individus avec leurs caractéristiques, écrire une fonction qui permet de normaliser par la moyenne et par l'écart-type.

- 1- Utiliser une fonction basée sur des boucles
- 2- Utiliser des fonctionnalités de numpy

Exercice 11:

Ecrire une fonction qui retourne la moyenne des valeurs paires et la moyennes des valeurs impaires. Utiliser des boucles et ensuite des fonctionnalités de numpy.

Ecrire deux versions:

- 1- une version à base de boucles
- 2- une version à base de fonctionnalités numpy

TP2, Décomposition LU, inversion d'une matrice, résolution de $Ax=B$

I. Problème.

On désire résoudre des systèmes de N équations linéaires, à N inconnues, systèmes que nous écrirons sous la forme matricielle : $AX = B$, où

- A est une matrice carrée à N lignes et N colonnes,
- X est le vecteur colonne solution cherché
- B est le vecteur appelé « membre droit »

Chaque ligne de l'équation matricielle ci-dessus peut s'écrire comme suit:

$$\sum_j a_{i,j} x_j = b_i.$$

II. Méthode de Crout

Elle repose sur la propriété suivante: si A est de rang n (c'est-à-dire le système admet une solution unique) alors il est possible d'écrire $A = LU$, où L est une matrice triangulaire inférieure et U une matrice triangulaire supérieure dont les termes diagonaux valent 1.

Calcul de L et U :

$$A = \begin{pmatrix} l_{1,1} & 0 & \dots & 0 & 0 \\ l_{2,1} & l_{2,2} & \dots & 0 & 0 \\ \vdots & \vdots & \ddots & & \\ l_{N,1} & l_{N,2} & \dots & l_{N,N-1} & 0 \\ l_{N,1} & l_{N,2} & \dots & l_{N,N-1} & l_{N,N} \end{pmatrix} \begin{pmatrix} 1 & u_{1,2} & \dots & u_{1,N-1} & u_{1,N} \\ 0 & 1 & \dots & u_{2,N-1} & u_{2,N} \\ \vdots & \vdots & \ddots & & \\ 0 & 0 & \dots & 1 & u_{N-1,N} \\ 0 & 0 & \dots & 0 & 1 \end{pmatrix}$$

Supposons calculées les $(h-1)$ première colonnes de L , et les $(h-1)$ premières lignes de U . Calculons la h^{eme} colonne de L et la h^{eme} ligne de U :

En prenant $j = h$, il vient:

$$a_{i,h} = \sum_{k=1}^h l_{i,k} \times u_{k,h} = l_{i,h} + \sum_{k=1}^{h-1} l_{i,k} \times u_{k,h}$$

Ce qui implique :

$$l_{i,h} = a_{i,h} - \sum_{k=1}^{h-1} l_{i,k} u_{k,h}, \text{ pour } i \geq h. \quad (Eq1)$$

Ce qui donne la h^{eme} colonne de L .

En prenant $i = h$ et $j > h$, il vient:

$$a_{h,j} = \sum_{k=1}^h l_{h,k} u_{k,j} = l_{h,h} u_{h,j} + \sum_{k=1}^{h-1} l_{h,k} u_{k,j}$$

$$u_{h,j} = \frac{a_{h,j} - \sum_{k=1}^{h-1} l_{h,k} u_{k,j}}{l_{h,h}}, \text{ pour } j \geq h. \quad (Eq2)$$

Ce qui donne la h^{eme} ligne de L .

Programmation :

Q1-1 Ecrire la fonction, qui étant donnée une matrice non singulière, renvoie les deux matrices L et U .

```
def LU_decomposition(A):
    L =
    U =
    Pour chaque h: // h ligne colonne
        mettre à jour la colonne h de L
        mettre à jour la ligne h de U
    retourner L et U
```

Q1-2 Vérifier que votre fonction marche.

Q1-3 Ecrire les équations $Eq1$ et $Eq2$ en utilisant une forme matricielle. En particulier, utiliser le fait qu'il est possible d'exprimer le calcul de la colonne h de L ou de la ligne h de U à l'aide d'une formule matricielle, sans recourir à des boucles. Intégrer ceci à votre fonction et tester.

Le plus grand pivot (en valeur absolue)

Dans ces calculs, pour calculer $u_{h,j}$ on doit diviser par $l_{h,h}$ qui peut être égal à zéro. Si la matrice A est non singulière alors il existe au moins un coefficient non nul dans la demie colonne de L . D'autre part, la précision des opérations est meilleure quand les diviseurs sont grands (en valeur absolue).

Après avoir calculer la l^{eme} colonne de L , vous devez trouver l'indice $m \geq l$ de la ligne qui contient, sur la même colonne, la plus grande valeur (en valeur absolue). Si $m \neq l$ alors échanger dans les matrices A et L les lignes m et l .

Une matrice d'échange de lignes est une matrice de permutation qui permet d'échanger deux lignes d'une matrice par multiplication à gauche. Pour permuter les lignes l et m d'une matrice carrée A , il suffit de multiplier à gauche A par la matrice P obtenue en échangeant les lignes l et m de la matrice identité I .

En python (numpy) voici la syntaxe pour échanger la ligne l et la ligne m d'une matrice A :

```
A[[l,m]] = A[[m,l]].
```

Dans la permutation de la m^{eme} ligne avec la l^{eme} ligne, il faut arrêter la permutation à la colonne $l-1$ (i.e les colonnes $[0 : l-1]$). En effet seulement ces colonnes sont supposées être déjà calculées. Attention après avoir échanger les lignes de A et de L , il

faut recalculer la colonne de L .

Q1-4 Ajouter cette procédure à votre fonction. Votre fonction doit retourner P (la matrice de permutation), la matrice L et la matrice U .

III. Inversion d'une triangulaire inférieure :

En utilisant le même principe de calcul, trouver les formules permettant de déterminer l'inverse d'une matrice triangulaire.

Soit L une matrice triangulaire inférieure :

$$L = \begin{pmatrix} l_{1,1} & 0 & \dots & 0 & 0 \\ l_{2,1} & l_{2,2} & \dots & 0 & 0 \\ \vdots & \vdots & \ddots & & \\ l_{N,1} & l_{N,2} & \dots & l_{N-1,N-1} & 0 \\ l_{N,1} & l_{N,2} & \dots & l_{N,N-1} & l_{N,N} \end{pmatrix}$$

Soit X l'inverse de L . On pose : $LX = I$, et on tente de déterminer X qui est une matrice carrée de même taille que L .

Ecrivons l'élément général de $LX = I$: $I_{i,j} = \sum_k l_{i,k} \times x_{k,j}$

Supposons que les $(h - 1)$ premières lignes sont calculées

$$\text{Pour } i = h: I_{h,j} = \sum_{k=0}^{k=h} l_{h,k} \times x_{k,j} = l_{h,h} \times x_{h,j} + \sum_{k=0}^{k=(h-1)} l_{h,k} \times x_{k,j}$$

Ce qui implique que :

$$x_{h,j} = \frac{I_{h,j} - \sum_{k=0}^{k=(h-1)} l_{h,k} \times x_{k,j}}{l_{h,h}}$$

Q2-1 Ecrire la fonction qui détermine l'inverse d'une matrice triangulaire inférieure

Q2-2 En déduire une fonction qui détermine l'inverse d'une matrice triangulaire supérieure. Pour cela utiliser la propriété suivante : $(A^T)^{-1} = (A^{-1})^T$

Q2-3 En déduire une fonction qui détermine l'inverse d'une matrice carrée (invertible). Utiliser le fait que $PA = PLU$ et que

Q2-4 En utilisant toute les fonctionnalités décrite ci-dessus, écrire une fonction qui renvoie la solution de l'équation $AX = B$. En suppose que A est invertible.

Forme échelonnée d'une matrice

TP3, L1 informatique

Une matrice est dite sous forme échelonnée si elle est sous la forme suivante:

$$\begin{pmatrix} * & * & * & * & * & * & * & * \\ 0 & 0 & * & * & * & * & * & * \\ 0 & 0 & 0 & * & * & * & * & * \\ 0 & 0 & 0 & 0 & 0 & 0 & * & * \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

Une matrice est dite sous forme échelonnée si :

1. toutes les lignes nulles sont situées en bas de la matrice
2. chaque ligne non nulle possède un pivot (premier coefficient non nul)
3. les pivots se déplacent strictement vers la droite d'une ligne à l'autre
4. tous les éléments situés sous un pivot sont nuls

Pour ce travail on considère comme nulle toute valeur x telle que $abs(x) < 1e - 10$.

1- On souhaite écrire une fonction qui prend en argument une matrice et qui renvoie vrai si la matrice est sous forme échelonnée, sinon elle renvoie faux.

1-a Écrire une fonction « trouver_pivot_sur_ligne » qui prend en argument une matrice A et un numéro de ligne i et qui renvoie le premier numéro de colonne j pour laquelle (valeur non nulle). Si aucune valeur non nulle n'est trouvée la fonction renvoie -1.

1-b Écrire une fonction « est_ligne_nulle » qui prend en argument une matrice A et un numéro de ligne i et qui renvoie vrai si la ligne i de A est entièrement nulle. Vous pouvez utiliser $np.all(np.abs(A[i]) < \epsilon)$.

1.c Écrire une fonction qui vérifie que toutes les lignes entièrement nulles sont situées en bas de la matrice.

1.d Écrire une fonction qui vérifie si toutes les valeurs en dessous d'un pivot sont nulles

1.e Écrire une fonction qui renvoie vrai si une matrice donnée est sous forme échelonnée et faux sinon.

La forme échelonnée d'une matrice peut avoir plusieurs applications. Par exemple, elle peut servir à déterminer le rang de la matrice. Il peut aussi déterminer lesquels parmi les vecteurs colonnes forment une famille libre. Lesquels parmi les vecteurs colonnes dépendent des autres vecteurs colonnes.

On se donne l'objectif de transformer une matrice quelconque en une matrice sous forme échelonnée. Voici l'algorithme que l'on souhaite implémenter:

```

Algorithme Echelonner(A, m, n)
  i ← 1
  j ← 1
  Tant que i ≤ m et j ≤ n faire
    k ← i
    Tant que k ≤ m et A[k,j] = 0 faire (boucle 1)
      k ← k + 1
    Fin Tant que
    Si k > m alors
      j ← j + 1
    Sinon
      Echanger la ligne i et la ligne k
      Pour r allant de i+1 à m faire (boucle 2)
        Si A[r,j] ≠ 0 alors
          facteur ← A[r,j] / A[i,j]
          Pour c allant de j à n faire
            A[r,c] ← A[r,c] - facteur × A[i,c]
          Fin Pour
        Fin Si
      Fin Pour
      i ← i + 1
      j ← j + 1
    Fin Si
  Fin Tant que
Fin
  
```

2- Ecrire la fonction Echelonner qui transforme la matrice M en une matrice sous forme échelonnée.

3- Prenez un exemple particulier pour lequel vous connaissez le résultat de l'échelonnement et faites exécuter votre fonction. Avez vous obtenu le résultat attendu ?

4- À l'aide de la réduction échelonnée de la matrice associée, déterminer une sous famille libre extraite de la famille . La sous famille doit être de cardinal maximal.

$$e_1 = \begin{pmatrix} 3 \\ 0 \\ 0 \\ 1 \end{pmatrix}, e_2 = \begin{pmatrix} 1 \\ 0 \\ 1 \\ 0 \end{pmatrix}, e_3 = \begin{pmatrix} 2 \\ 1 \\ 0 \\ 5 \end{pmatrix}, e_4 = \begin{pmatrix} 3 \\ 2 \\ 1 \\ 0 \end{pmatrix}, e_5 = \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \end{pmatrix}, e_6 = \begin{pmatrix} 1 \\ 0 \\ 1 \\ 2 \end{pmatrix}$$

Est ce que la famille trouvée est une base de $Vect(e_1, e_2, e_3, e_4, e_5, e_6)$?

Dites quels sont les vecteurs qui s'écrivent comme combinaison linéaires des autres.

5- Nous souhaitons éliminer le maximum de boucles dans la fonction. Afin de remplacer la boucle 1 par une opération, utiliser `np.where` pour extraire tous les indices du vecteur colonne $A[i : , j]$ dont la valeur absolue est supérieure à ϵ (i.e valeurs nulles):

```
pivot_candidats=np.where(np.abs(A[i:, j])>epsilon))[0]
```

Remarquez que dans la boucle 2, pour chaque ligne i et pour chaque colonne j , pour chaque ligne l à partir de $i+1$ et pour chaque colonne c à partir j , $A[l,c]$ est égale à $A[l,c]$ moins $A[i,c] \times A[l,j]/A[i,j]$.

On peut implémenter ça en faisant (seulement si $i+1 < m$) :

```
f = A[i+1:,j]/A[i,j]
```

```
A[i+1:,j:]=f[:,None] * A[i,j:]
```

Vous pouvez aussi faire : $A[\text{np.abs}(A) < \epsilon] = 0.0$.

TP4, numpy

1- Soit le système suivant:

$$\begin{cases} v - w = 3 \\ -2u + 4v - w = 1 \\ -2u + 5v - 4w = -2 \end{cases}$$

1- Sur papier, utiliser l'élimination de Gauss pour résoudre le système

2- Utiliser `x = np.linalg.solve(A, B)` pour résoudre le système. Est ce que vous trouvez le même résultat ?

2-

Soit les deux matrices

$$A = \begin{pmatrix} 9 & -6 & 10 \\ -5 & 2 & -5 \\ -12 & 6 & -13 \end{pmatrix} \text{ et } B = \begin{pmatrix} 9 & -6 & 10 \\ -5 & 2 & -5 \\ -12 & 6 & -13 \end{pmatrix}$$

1- Calculer à la main AB .

2- Ecrire une fonction qui retourne le produit scalaire de deux matrices lignes

3- Ecrire une fonction qui prend en argument deux matrices et qui renvoie le produit, en utilisant la fonction du 2-

4- Comparer les temps de calcul du 3 avec `C = np.dot(A, B)` sur des matrices de très grandes tailles (voir tp précédent)

3-

Soit une matrice A de valeurs réels contenant les notes d'étudiants dans différentes matières. Les lignes correspondent aux étudiants de la L1 informatique, par exemple et les colonnes correspondent aux notes dans différentes matières. Par exemple, l'étudiant qui se trouve à la ligne e admet comme note $A[e, m]$ dans la matière qui retrouve à la colonne m .

1 Ecrire une fonction qui renvoie:

- la moyenne et l'écart-type de tous les étudiants dans toutes les matières
- les moyennes et les écart-types de tous les étudiants par matière

2- Ecrire une fonction qui renvoie les indices de la matrice pour lesquelles la moyenne est supérieure à une valeurs donnée.

4-

- 1- Créer une matrice A de taille 5×5
- 2- Calculer son inverse (avec une fonction de numpy)
- 3- Vérifier que $A \times A^{-1} = I_5$

5-

```
import numpy as np

def chercher_pivot(M, l, c):
    pivot = l
    for k in range(l, M.shape[0]):
        if abs(M[k][c]) > abs(M[pivot][c]):
            pivot = k
    return pivot

def echange_lignes(M, i, j):
    M[i], M[j] = M[j].copy(), M[i].copy()

def transvection(M, i, j, c):
    M[j] = M[i, c] * M[j] - M[j, c] * M[i]

def echelonner(A):
    n1 = A.shape[0]
    n2 = A.shape[1]
    l = 0
    for c in range(n2):
        if l == (n1 - 1):
            break
        pivot = chercher_pivot(A, l, c)
        if (A[pivot][c] == 0):
            continue
        if pivot > l:
            echange_lignes(A, l, pivot)
        for k in range(l + 1, n1):
            transvection(A, l, k, c)
        l = l + 1
```

- 1- Ecrire le code ci-dessus et tester le sur les matrices:

```
A = np.array([[1, 2, 1, 1], [2, 4, 2, 2], [3, 6, 3, 4]], dtype=float)
B = A = np.array([[1, 2, 1, 3, 3], [2, 4, 0, 4, 4], [1, 2, 3, 5, 5],
                  [2, 4, 0, 4, 7]], dtype=float)
```

- 2- Modifier le code pour que la fonction `echelonner` renvoie le rang de la matrice. Le rang d'une matrice est le nombre (max) de vecteurs colonnes qui forment une famille libre. C'est aussi le nombre de pivot (non nuls forcément).

- 3- En fait, la famille libre de taille maximale est celle composée des vecteurs colonnes correspondants à des pivots (non nuls) dans la forme échelonnée de la matrice initiale. Modifier la fonction `echelonner` pour qu'elle renvoie une matrice dont les vecteurs colonnes forment la sous-famille libre de taille maximale